Airflow - Advanced DAGs

Recurring DAGs:

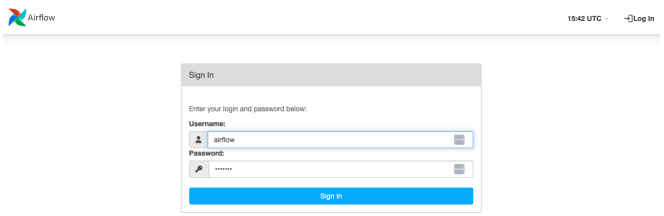
Create a DAG minutely_dag.py containing a task printing time and day, set DAG to run every minute

minutely_dag.py U

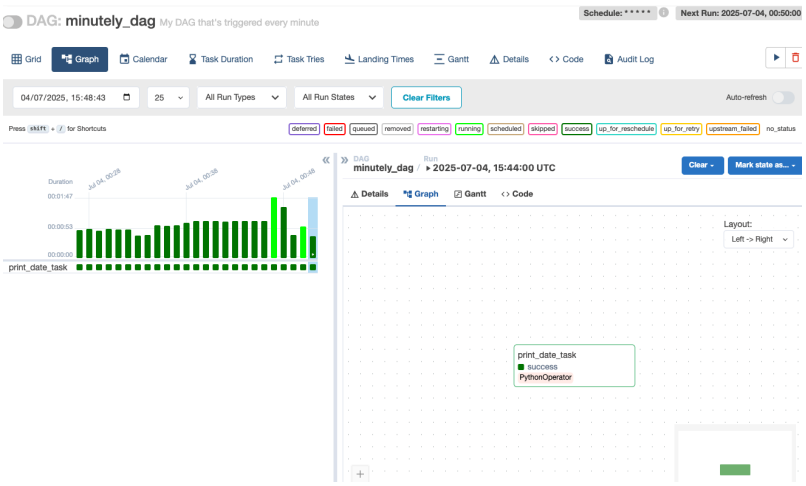
datascientest-data-engineering > airflow > dags > minutely_dag.py

```python
from airflow import DAG
from airflow.utils.dates import days_ago
from airflow.operators.python import PythonOperator
import datetime


def print_date():
    print(datetime.datetime.now())

with DAG(
    dag_id = 'minutely_dag',
    description = 'My DAG that\'s triggered every minute',
    tags = ['tutorial', 'datascientest'],
    schedule_interval = '* * * * *',
    default_args = {
        'owner': 'airflow',
        'start_date': days_ago(0, minute=1),
    }
) as my_dag:

    my_task = PythonOperator(
        task_id = 'print_date_task',
        python_callable = print_date
    )
```

Open URL http://52.48.204.31:8080/ with username airflow and password airflow



After clicking on play button to trigger, open Graph view



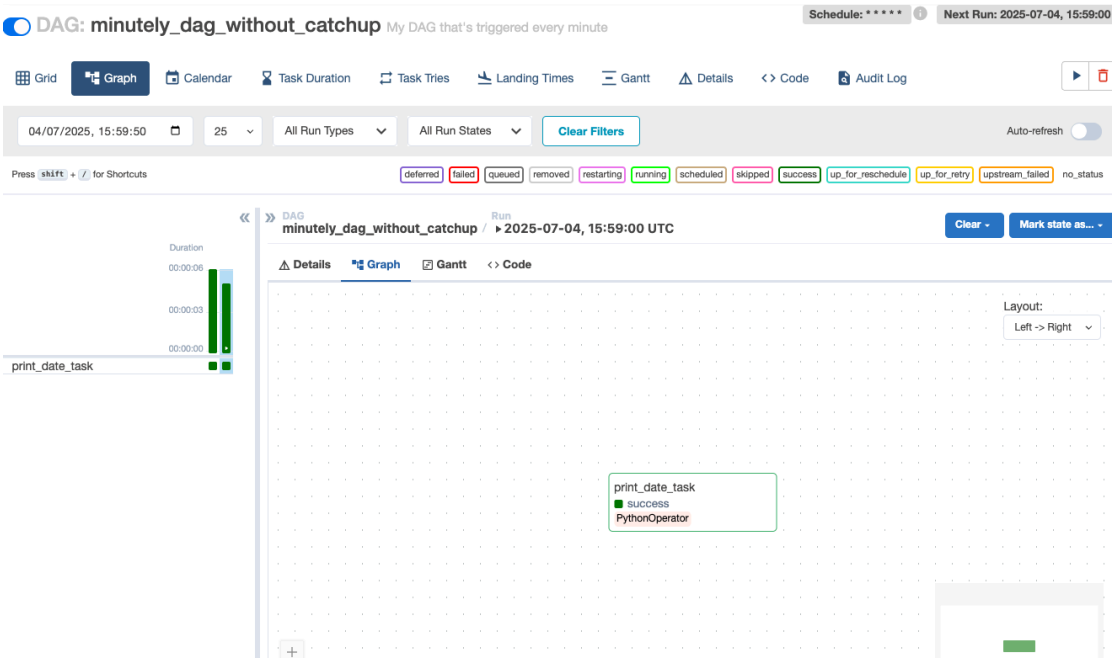
Press pause, then edit the minutely_dag.py file

minutely_dag.py U

datascientest-data-engineering > airflow > dags > minutely_dag.py

```python
from airflow import DAG
from airflow.utils.dates import days_ago
from airflow.operators.python import PythonOperator
import datetime


def print_date():
    print(datetime.datetime.now())

with DAG(
    dag_id = 'minutely_dag_without_catchup',
    description = 'My DAG that\'s triggered every minute',
    tags = ['tutorial', 'datascientest'],
    schedule_interval = '* * * * *',
    default_args = {
        'owner': 'airflow',
        'start_date': days_ago(0, minute=1),
    },
    catchup=False
) as my_dag:

    my_task = PythonOperator(
        task_id = 'print_date_task',
        python_callable = print_date
    )
```

Open Airflow UI again, search for DAG minutely_dag and click play button to trigger



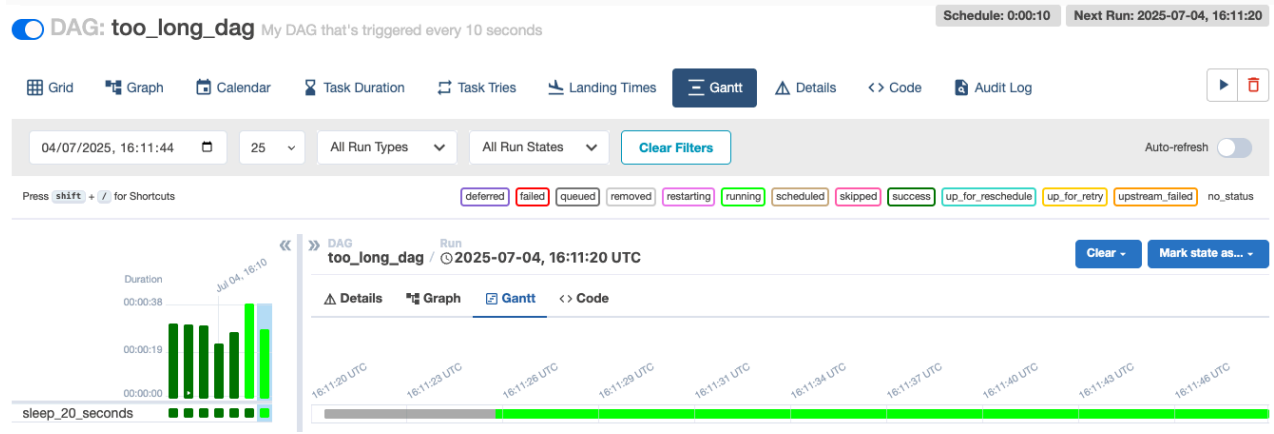
Create a DAG too_long_dag.py, this DAG takes longer to execute than expected schedule_interval

too_long_dag.py U

datascientest-data-engineering > airflow > dags > too_long_dag.py

```python
from airflow import DAG
from airflow.utils.dates import days_ago
from airflow.operators.python import PythonOperator
import datetime
import time


def sleep_20_seconds():
    time.sleep(20)

with DAG(
    dag_id = 'too_long_dag',
    description = 'My DAG that\'s triggered every 10 seconds',
    tags = ['tutorial', 'datascientest'],
    schedule_interval = datetime.timedelta(seconds=10),
    default_args = {
        'owner': 'airflow',
        'start_date': days_ago(0, minute=1),
    },
    catchup = False
) as my_dag:

    my_task = PythonOperator(
        task_id = 'sleep_20_seconds',
        python_callable = sleep_20_seconds
    )
```

Open Airflow UI, search for DAG too_long_dag and click play button to trigger. Note: DAG takes at least 20 seconds to run, whereas it launches every 10 seconds.



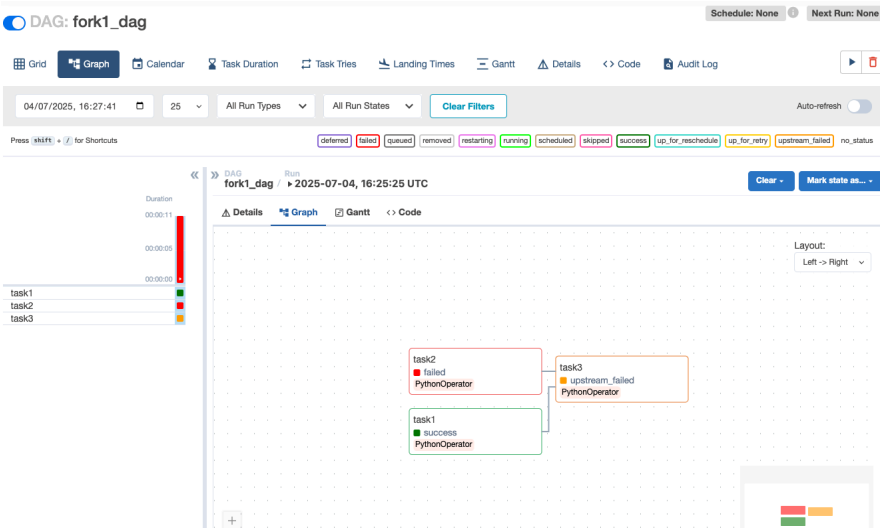
Conditional workflows:
Create DAG fork1_dag.py, a simple fork DAG to demo trigger_rule.

fork1_dag.py U

tascientest-data-engineering > airflow > dags > fork1_dag.py

```python
import random
from airflow import DAG
from airflow.utils.dates import days_ago
from airflow.operators.python import PythonOperator


def successful_task():
    print('success')

def failed_task():
    raise Exception('This task did not work!')

def random_fail_task():
    random.seed()
    if random.random() < .9:
        raise Exception('This task randomly failed')

with DAG(
    dag_id = 'fork1_dag',
    tags = ['tutorial', 'datascientest'],
    schedule_interval = None,
    default_args = {
        'owner': 'airflow',
        'start_date': days_ago(0, minute=1)
    },
    catchup = False
) as my_dag:
    task1 = PythonOperator(
        task_id = 'task1',
        python_callable = successful_task
    )

    task2 = PythonOperator(
        task_id = 'task2',
        python_callable = failed_task
    )

    task3 = PythonOperator(
        task_id = 'task3',
        python_callable = successful_task
    )

    [task1, task2] >> task3
```

Open Airflow UI again, search for DAG fork1_dag and click play button to trigger. DAG cannot execute correctly as task2 task will inevitably fail and prevent task3 task from executing. This behaviour is due to the default value of the trigger_rule argument.



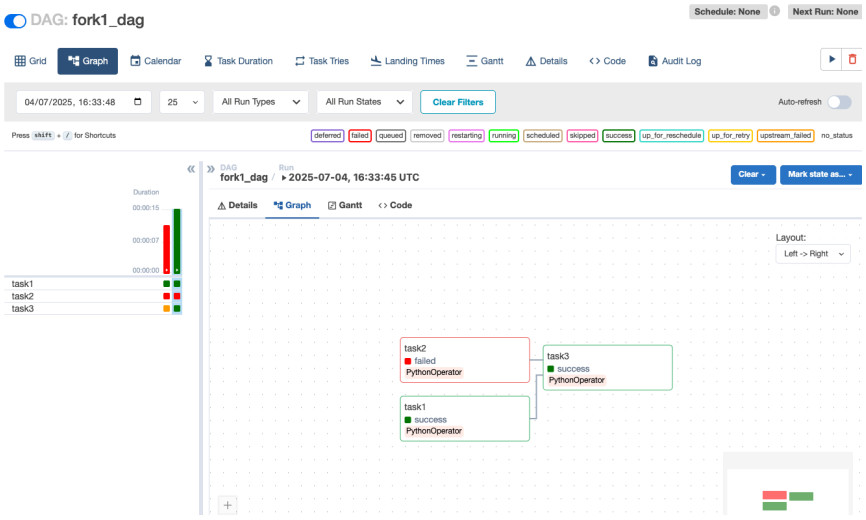
Edit file fork1_dag.py to fix with trigger_rule='all done' to change the task3 task

fork1_dag.py U

datascientest-data-engineering > airflow > dags > fork1_dag.py

```python
import random
from airflow import DAG
from airflow.utils.dates import days_ago
from airflow.operators.python import PythonOperator


def successful_task():
    print('success')

def failed_task():
    raise Exception('This task did not work!')

def random_fail_task():
    random.seed()
    if random.random() < .9:
        raise Exception('This task randomly failed')

with DAG(
    dag_id = 'fork1_dag',
    tags = ['tutorial', 'datascientest'],
    schedule_interval = None,
    default_args = {
        'owner': 'airflow',
        'start_date': days_ago(0, minute=1)
    },
    catchup = False
) as my_dag:
    task1 = PythonOperator(
        task_id = 'task1',
        python_callable = successful_task
    )

    task2 = PythonOperator(
        task_id = 'task2',
        python_callable = failed_task
    )

    task3 = PythonOperator(
        task_id = 'task3',
        python_callable = successful_task,
        trigger_rule = 'all_done'
    )

    [task1, task2] >> task3
```

Open Airflow UI and search again for DAG fork1_dag and click play button to trigger (again). Task3 task is now successfully running green.



Create a full conditional “branching” DAG conditional_dag.py which takes a schedule_interval of 10 seconds to be able to see the different behaviours.

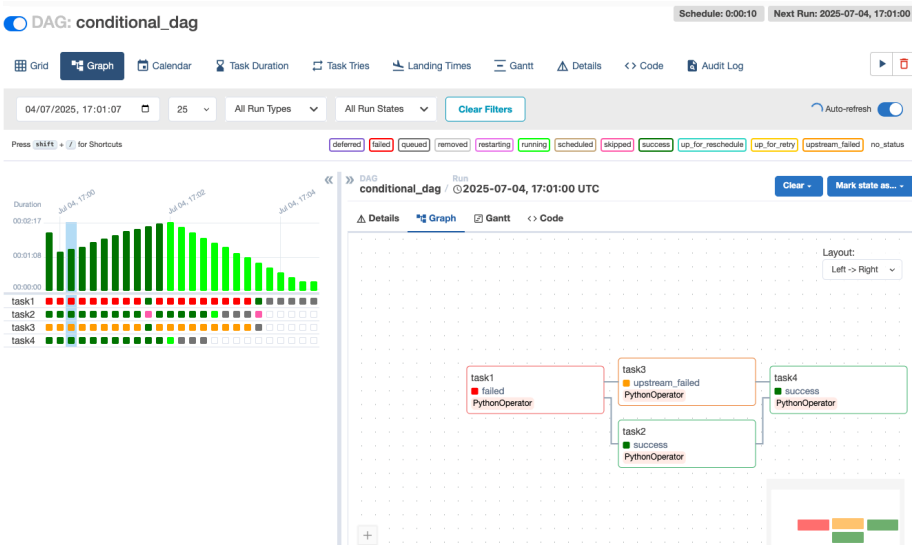
conditional_dag.py U

tascientest-data-engineering > airflow > dags > conditional_dag.py

```python
from airflow import DAG
from airflow.utils.dates import days_ago
from airflow.operators.python import PythonOperator
import random
import datetime


def successful_task():
    print('success')

def failed_task():
    raise Exception('This task did not work!')

def random_fail_task():
    random.seed()
    a = random.random()
    print(a)
    if a < .9:
        raise Exception('This task randomly failed')

with DAG(
    dag_id = 'conditional_dag',
    tags = ['tutorial', 'datascientest'],
    schedule_interval = datetime.timedelta(seconds=10),
    default_args = {
        'owner': 'airflow',
        'start_date': days_ago(0, minute=1)
    },
    catchup = False
) as my_dag:
    task1 = PythonOperator(
        task_id = 'task1',
        python_callable = random_fail_task
    )

    task2 = PythonOperator(
        task_id = 'task2',
        python_callable = successful_task,
        trigger_rule = 'all_failed'
    )

    task3 = PythonOperator(
        task_id = 'task3',
        python_callable = successful_task,
        trigger_rule = 'all_success'
    )
```

```python

    task4 = PythonOperator(
        task_id = 'task4',
        python_callable = successful_task,
        trigger_rule = 'all_done'
    )

    task1 >> [task2, task3]
    [task2, task3] >> task4

```

Open Airflow UI and search for DAG conditional_dag and click play button to trigger.



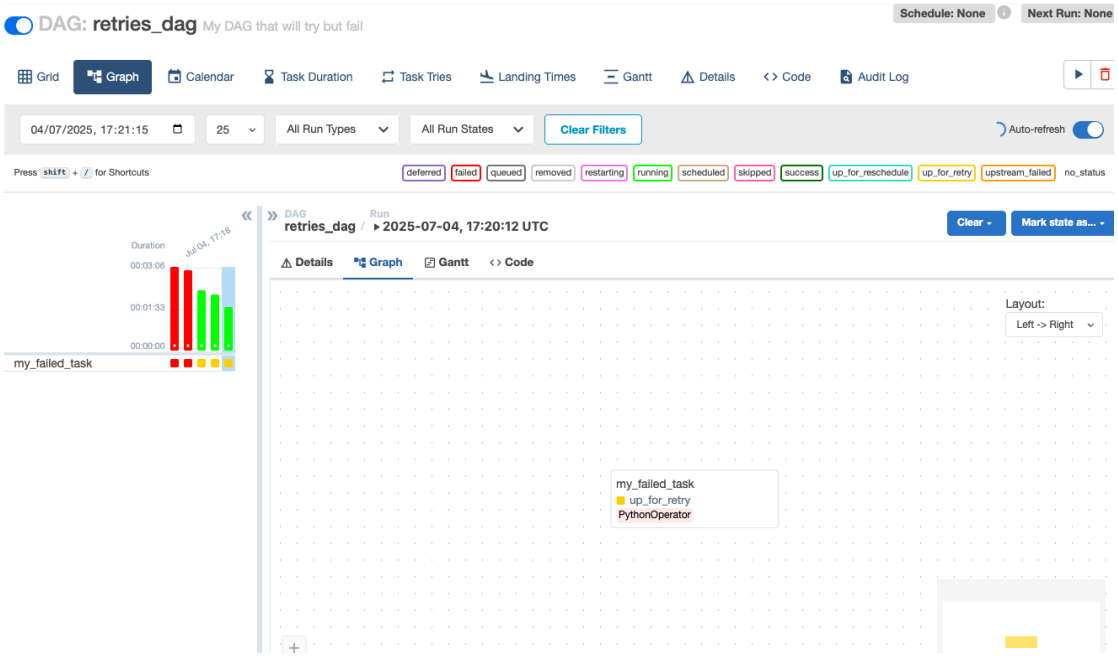
New attempts:
For the purposes of retries on failure create DAG retries_dag.py

retries_dag.py U

tascientest-data-engineering > airflow > dags > retries_dag.py

```python
from airflow import DAG
from airflow.utils.dates import days_ago
from airflow.operators.python import PythonOperator
import datetime


def failed_task():
    raise Exception('This task did not work!')

with DAG(
    dag_id = 'retries_dag',
    description = 'My DAG that will try but fail',
    tags = ['tutorial', 'datascientest'],
    schedule_interval = None,
    default_args = {
        'owner': 'airflow',
        'start_date': days_ago(0, minute=1),
    },
    catchup = False
) as my_dag:
    task1 = PythonOperator(
        task_id = "my_failed_task",
        python_callable = failed_task,
        retries = 5,
        retry_delay = datetime.timedelta(seconds=30)
    )
```

Open Airflow UI and search for DAG retries_dag and click play button to trigger.



Setup and Teardown:
Create a setup_and_teardown_dag.py for setup and teardown hooks

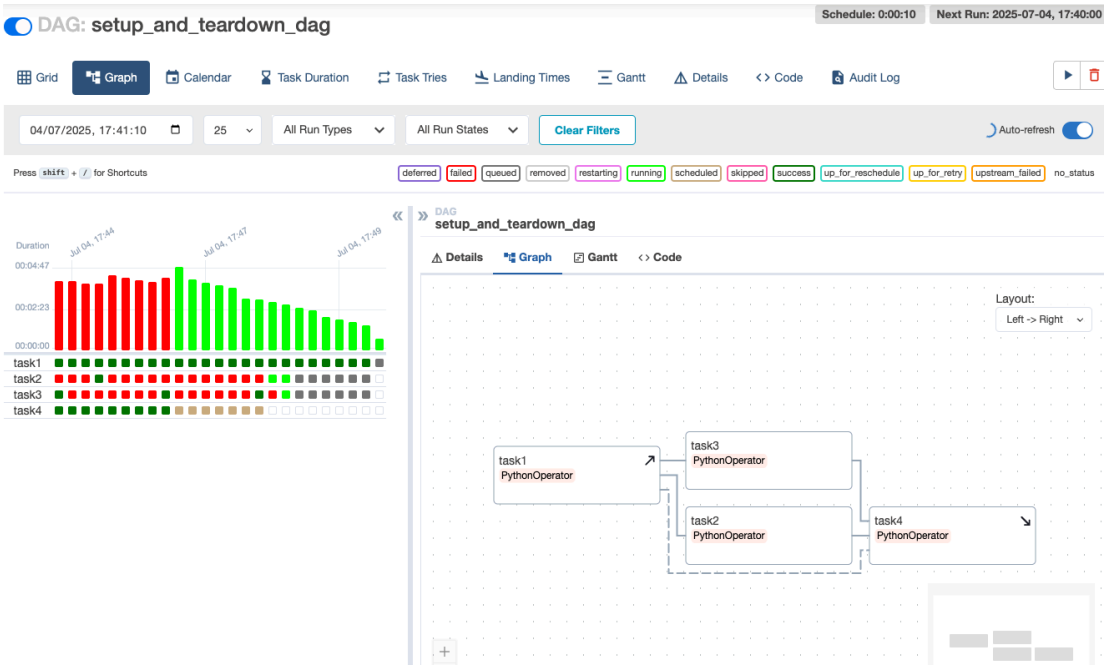
setup_and_teardown_dag.py U

tascientest-data-engineering > airflow > dags > setup_and_teardow

```python
from airflow import DAG
from airflow.utils.dates import days_ago
from airflow.operators.python import PythonOperator
import random
import datetime


def successful_task():
    print('success')

def failed_task():
    raise Exception('This task did not work!')

def random_fail_task():
    random.seed()
    a = random.random()
    print(a)
    if a < .9:
        raise Exception('This task randomly failed')

with DAG(
    dag_id = 'setup_and_teardown_dag',
    tags = ['tutorial', 'datascientest'],
    schedule_interval = datetime.timedelta(seconds=10),
    default_args = {
        'owner': 'airflow',
        'start_date': days_ago(0, minute=1)
    },
    catchup = False
) as my_dag:
    task1 = PythonOperator(
        task_id = 'task1',
        python_callable = successful_task
    )

    task2 = PythonOperator(
        task_id = 'task2',
        python_callable = random_fail_task
    )

    task3 = PythonOperator(
        task_id = 'task3',
        python_callable = random_fail_task
    )

    task4 = PythonOperator(
        task_id = 'task4',
        python_callable = successful_task
    )
```

```python
    setup = task1.as_setup()
    teardown = task4.as_teardown()
    setup >> [task2, task3]
    [task2, task3] >> teardown
    task1 >> task4
```

Open Airflow UI and search for DAG setup_and_teardown_dag and click play button to trigger.



Notifications:
Create a notifications_dag.py DAG. Add email in .env file for security purposes.

notifications_dag.py U

tascientest-data-engineering > airflow > dags > notifications_dag.py

```python
from airflow import DAG
from airflow.utils.dates import days_ago
from airflow.operators.python import PythonOperator
import os
import datetime


def failed_task():
    raise Exception("This task did not work!")

# email address in .env
ALERT_EMAIL = os.environ.get("ALERT_EMAIL")

with DAG(
    dag_id = "notifications_dag",
    description = "Retries + email notifications on failure",
    schedule_interval = None,
    catchup = False,
    default_args = {
        "owner": "airflow",
        "start_date": days_ago(0, minute=1),
    },
    tags = ["tutorial", "datascientest"]
) as dag:

    task1 = PythonOperator(
        task_id = "my_failed_task",
        python_callable = failed_task,
        retries = 5,
        retry_delay = datetime.timedelta(seconds=30),

        email_on_retry = True,
        email_on_failure = True,
        email = [ALERT_EMAIL]
    )
```

Update docker-compose.yaml file to include **ALERT_EMAIL: “${ALERT_EMAIL}”**

Restart the stack

```
ubuntu@ip-172-31-46-112:~/datascientest-data-engineering/airflow$ docker-compose down
Stopping airflow_flower_1            ... done
Stopping airflow_airflow-webserver_1 ... done
Stopping airflow_airflow-scheduler_1 ... done
Stopping airflow_airflow-worker_1    ... done
Stopping airflow_postgres_1          ... done
Stopping airflow_redis_1             ... done
Removing airflow_flower_1            ... done
Removing airflow_airflow-webserver_1 ... done
Removing airflow_airflow-scheduler_1 ... done
Removing airflow_airflow-worker_1    ... done
Removing airflow_airflow-init_1      ... done
Removing airflow_postgres_1          ... done
Removing airflow_redis_1             ... done
Removing network airflow_default
ubuntu@ip-172-31-46-112:~/datascientest-data-engineering/airflow$ docker-compose up -d
Building with native build. Learn about native build in Compose here: https://docs.docker.com/go/compose-native-build/
Creating network "airflow_default" with the default driver
Creating airflow_redis_1    ... done
Creating airflow_postgres_1 ... done
Creating airflow_airflow-worker_1    ... done
Creating airflow_airflow-scheduler_1 ... done
Creating airflow_airflow-init_1      ... done
Creating airflow_flower_1            ... done
Creating airflow_airflow-webserver_1 ... done
ubuntu@ip-172-31-46-112:~/datascientest-data-engineering/airflow$
```

Open Airflow UI, search for notifications_dag and trigger DAG by selecting the play button.